

Εισαγωγή Αναφοράς:

Σκοπός της άσκησης ήταν η δημιουργία και η σύγκριση 4 δομών δεδομένων (2 στην μνήμη και 2 στον δίσκο) για την εύρεση του πιο αποδοτικού τρόπου αναζήτησης στοιχείων. Τα στοιχεία που χρησιμοποιήθηκαν ήταν ζευγάρια ακεραίων σε εύρος $N = 2^{18}$. Η κάθε σελίδα στον δίσκο καταλάμβανε 256 bytes, ενώ το μέγεθος των πινάκων (Hash) που χρησιμοποιήθηκαν ήταν $M = 100$.

A.1) Συνδεδεμένες λίστες:

Για την υλοποίηση της συνδεδεμένης λίστας με στοιχεία (x,y) χρησιμοποιήθηκαν τα interfaces που δόθηκαν στο φροντιστήριο του μαθήματος με ορισμένες αλλαγές. Αυτά ήταν τα Point, Search, MemListItem και MemList που υλοποιήθηκαν με τις κλάσεις Position, PosNode και MemoryList. Έγινε, λοιπόν, εισαγωγή στοιχείου στο τέλος της λίστας, με τη βοήθεια των pointers head και tail, όπου κάθε φορά σημαντικός ήταν ο έλεγχος για άδεια λίστα. Επίσης, έγινε αναζήτηση στοιχείου διατρέχοντας σειριακά όλα τα στοιχεία της.

A.2) Μέθοδος κατακερματισμού:

Σε αυτή τη μέθοδο χρησιμοποιήθηκαν οι παραπάνω συνδεδεμένες λίστες με τη διαφορά ότι αποτέλεσαν στοιχεία ενός πίνακα διάστασης M . Η εισαγωγή και η αναζήτηση στοιχείων έγιναν όπως και πριν, ενώ για να επιλεγεί η θέση της λίστας στον πίνακα χρησιμοποιήθηκε η συνάρτηση που δόθηκε στην εκφώνηση: $H(x, y) = (x * N + y) \% M$. Βοηθητικό ήταν το interface MemHash, το οποίο και υλοποιήθηκε με την κλάση MemoryHash.

B.1) Συνδεδεμένες λίστες στον δίσκο:

Για την δημιουργία αυτής της δομής δεδομένων, χρησιμοποιήθηκαν τα interfaces DataPage (υλοποίηση με την ListPage) και FileHandling (υλοποίηση με την DiskList), τα οποία βοήθησαν στην μεταφορά σελίδων από και προς τον δίσκο και την επεξεργασία τους στην μνήμη. Πιο αναλυτικά, δημιουργήθηκε ένα αρχείο, στο οποίο έγιναν πολλές εισαγωγές στοιχείων ως εξής: Κάθε ListPage στη μνήμη γέμιζε με στοιχεία ($256/8 = 32$ maximum) και έπειτα γινόταν η εγγραφή της στο αρχείο. Η τελευταία σελίδα, είτε γέμιζε, είτε όχι, γραφόταν στο αρχείο ούτως ή άλλως. Για την αναζήτηση στοιχείου διαβάστηκε κάθε σελίδα του αρχείου σειριακά και ελέγχθηκε κάθε στοιχείο της, έως ότου βρεθεί το επιθυμητό στοιχείο ή έως ότου τελειώσουν οι

σελίδες του αρχείου. Στο τέλος το αρχείο διαγράφηκε, έτσι ώστε να μην υπάρξει πρόβλημα σε περίπτωση που επαναληφθεί η παραπάνω διαδικασία.

B.2) Μέθοδος κατακερματισμού στον δίσκο:

Σε αυτή τη μέθοδο οι εισαγωγές και οι αναζητήσεις στο αρχείο δεν έγιναν με σειριακό τρόπο. Συνεπώς, η κάθε σελίδα του αρχείου χρειάστηκε να αποθηκεύει στην αρχή της τον αριθμό των στοιχείων που περιλαμβάνει (31 maximum) και τον αριθμό (θέση) της επόμενης σελίδας της λίστας. Πρακτικά, χρησιμοποιήθηκαν δύο νέες κλάσεις, οι HashPage και DiskHash, οι οποίες όμως υιοθέτησαν ή επαναόρισαν ορισμένες μεταβλητές και μεθόδους των ListPage και DiskList αντίστοιχα, και άρα έγιναν υποκλάσεις τους.

Αρχικά, είχε δημιουργηθεί ένας δισδιάστατος πίνακας $2 \times M$, στον οποίο αποθηκεύονταν οι θέσεις στο αρχείο της πρώτης και της τελευταίας σελίδας της κάθε λίστας σελίδων. Ωστόσο, σε 2^η φάση έγινε η υλοποίηση του bonus (κλάση bonusFile), με την αποθήκευση του πίνακα σε ένα ξεχωριστό αρχείο. Το αρχείο αυτό δεν ήταν κατανεμημένο σε σελίδες των 256 bytes, καθώς αυτή τη φορά ήταν γνωστό εκ των προτέρων το μέγεθός του και το μέγεθος των στοιχείων του, τα οποία είχαν εισαχθεί σε αυτό με βάση την συνάρτηση $H(x, y) = (x * N + y) \% M$. Επομένως, για κάθε διάβασμα ή γράψιμο ενός στοιχείου (αποτελούμενο από 2 integers head και tail) στο αρχείο έγινε seek στην θέση $H(x, y) * 8$. Να σημειωθεί ότι πριν τις εισαγωγές, τα στοιχεία του bonus αρχείου αρχικοποιήθηκαν στο -1.

Όσο αφορά στην λογική των δύο βασικών συναρτήσεων (insert και search), αυτή ήταν η εξής: Για κάθε εισαγωγή στοιχείου, αφού βρέθηκε η επιθυμητή θέση της λίστας σελίδων και διαβάστηκαν τα head και tail, έγινε ο έλεγχος για το αν η λίστα είναι άδεια, ώστε να αρχικοποιηθεί με νέα σελίδα. Διαφορετικά, η εισαγωγή του στοιχείου έγινε στην τελευταία σελίδα (tail), όπου σε περίπτωση που ήταν γεμάτη, δημιουργήθηκε νέα. Σε κάθε περίπτωση έγινε ενημέρωση των head και tail στο bonus αρχείο και γράψιμο της σελίδας στο βασικό αρχείο. Για την αναζήτηση στοιχείου, μετά την εύρεση της θέσης της λίστας, έγινε διάσχιση της σειριακά και με κάθε διάβασμα σελίδας έγινε σύγκριση όλων των στοιχείων της με το επιθυμητό στοιχείο. Για άλλη μια φορά ελέγχθηκε το ενδεχόμενο άδειας λίστας και στο τέλος των αναζητήσεων το αρχείο διαγράφηκε.

Γ) Σύγκριση μεθόδων και τεκμηρίωση:

Για τις ζητούμενες μετρήσεις χρησιμοποιήθηκε ο μετρητής MultiCounter που δόθηκε στα φροντιστήρια. Για τις αναζητήσεις στη μνήμη μετρήθηκε ο αριθμός συγκρίσεων και αναθέσεων, ενώ για τις αναζητήσεις στον δίσκο μετρήθηκε ο αριθμός των προσβάσεων. Στην κλάση Console, λοιπόν, παράχθηκαν τυχαίοι αριθμοί εύρους από 0 έως 2^{18} , οι οποίοι εισήχθησαν σε ζευγάρια στις δομές δεδομένων που υλοποιήθηκαν παραπάνω. Ο αριθμός των εισαγωγών καθορίστηκε

από το εκάστοτε K (K = 1.000, 10.000, 30.000, 50.000, 70.000, 100.000). Εν συνεχεία, για κάθε K έγιναν 100 επιτυχημένες και 100 αποτυχημένες αναζητήσεις στην κάθε δομή. Τα στοιχεία που αναζητήθηκαν παράχθηκαν από την ίδια τυχαία γεννήτρια, μόνο που για τις επιτυχημένες χρησιμοποιήθηκαν στοιχεία που είχαν εισαχθεί στην δομή, ενώ για τις αποτυχημένες χρησιμοποιήθηκαν στοιχεία εύρους από $2^{18}+1$ έως $2^{18}+101$. Τέλος, βρέθηκε ο μέσος όρος κάθε αναζήτησης και τα αποτελέσματα εκτυπώθηκαν στην οθόνη ως εξής:

 Javadoc

 Declaration

 Console

<terminated>

Console [Java Application] C:\Users\elena\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w

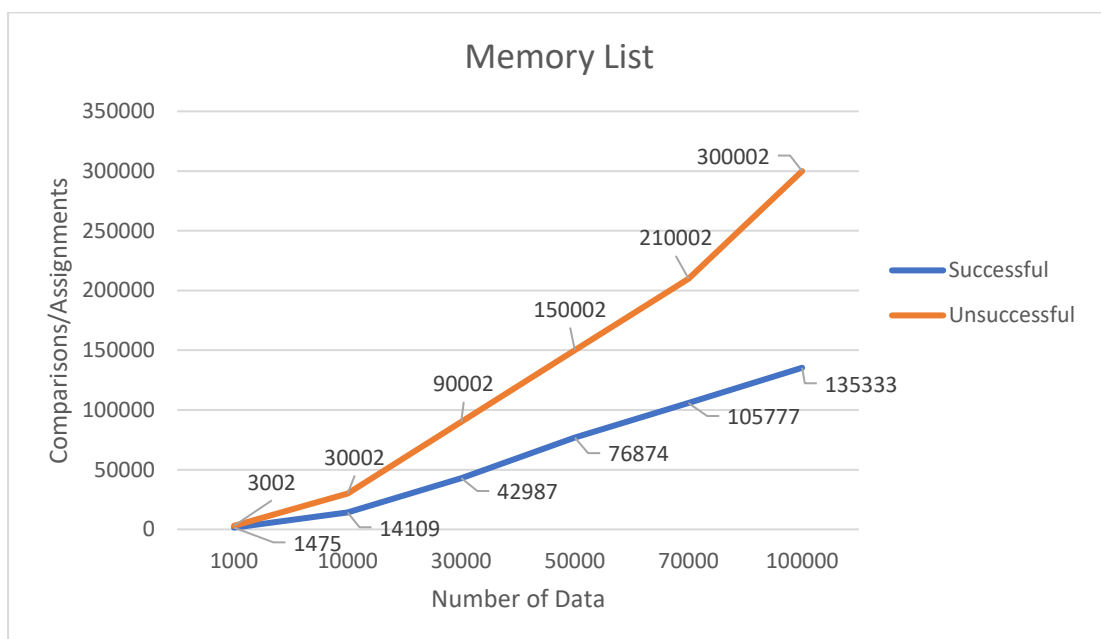
Successful searches:

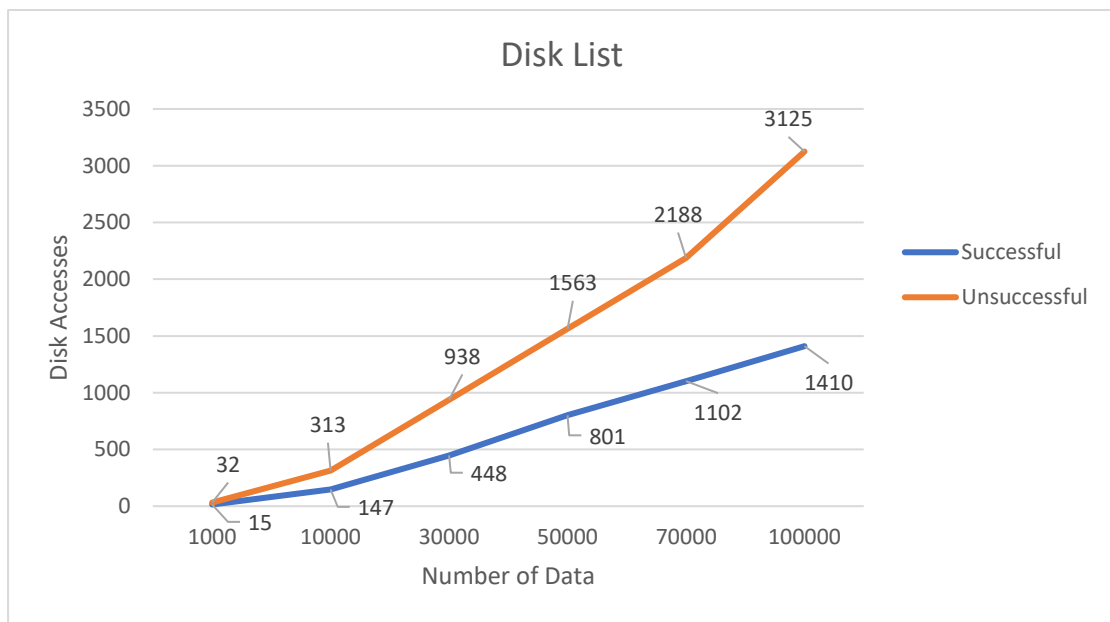
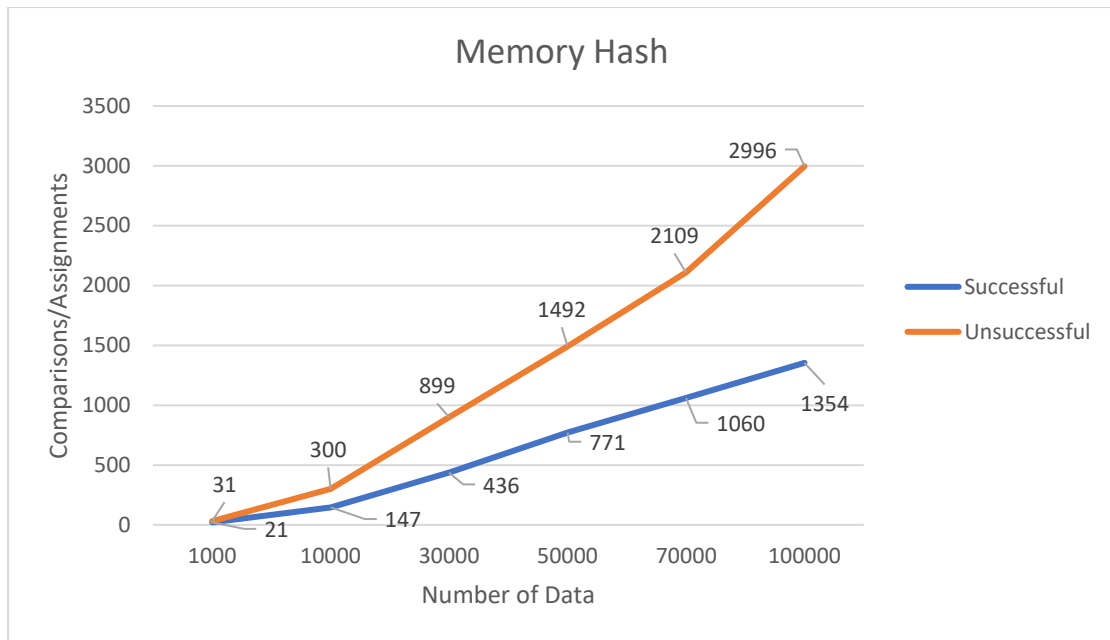
Number of Data	Memory List	Memory Hash	Disk List	Disk Hash
1000	1475	21	15	1
10000	14109	147	147	2
30000	42987	436	448	5
50000	76874	771	801	8
70000	105777	1060	1102	11
100000	135333	1354	1410	15

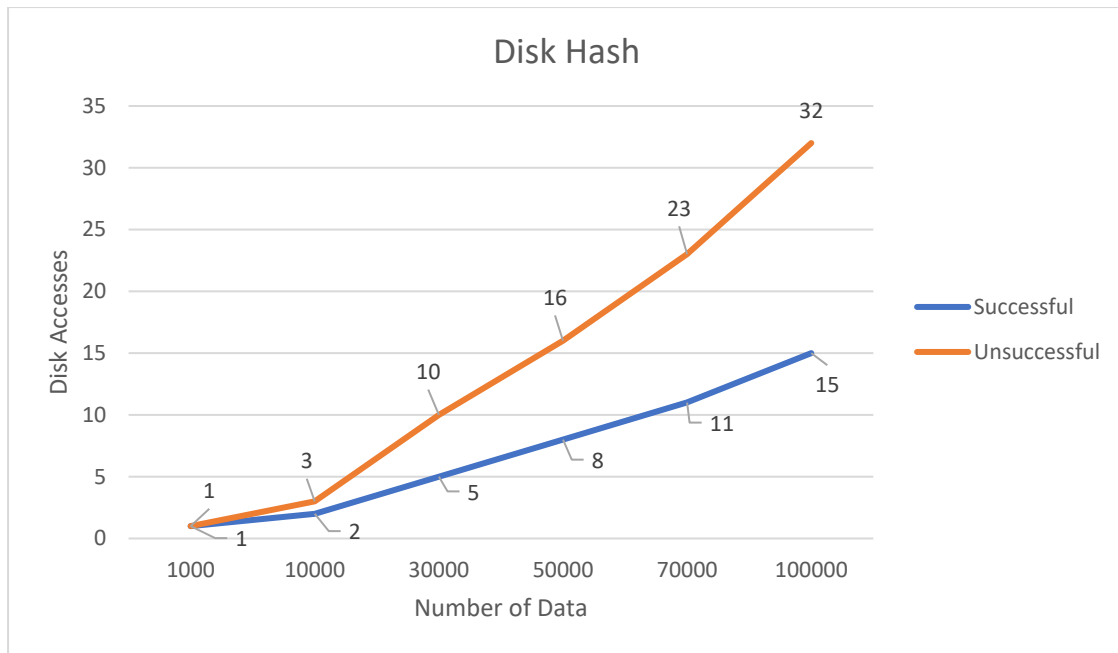
Unsuccessful searches:

Number of Data	Memory List	Memory Hash	Disk List	Disk Hash
1000	3002	31	32	1
10000	30002	300	313	3
30000	90002	899	938	10
50000	150002	1492	1563	16
70000	210002	2109	2188	23
100000	300002	2996	3125	32

Με βάση αυτές τις τιμές σχεδιάστηκαν σε κοινό διάγραμμα οι μετρήσεις των επιτυχημένων και των αποτυχημένων αναζητήσεων για την κάθε δομή, όπως φαίνεται παρακάτω:







Συμπερασματικά, είτε πρόκειται για συγκρίσεις και αναθέσεις στην μνήμη, είτε για προσβάσεις στον δίσκο, η μέθοδος του κατακερματισμού είναι αποδοτικότερη και συγκεκριμένα οι αριθμοί είναι 100 φορές μικρότεροι κατά προσέγγιση, έχοντας επιλέξει μέγεθος πίνακα $M = 100$. Παρατηρήθηκε ότι όσο μικρότερο ήταν το μέγεθος του πίνακα, τόσο μεγαλύτερες ήταν οι μετρήσεις. Γενικότερα με αυτή τη μέθοδο μοιράστηκαν τα δεδομένα σε πολλές λίστες, οπότε και η αναζήτησή τους ήταν ευκολότερη από το να υπήρχε μία ενιαία λίστα.

Όσο αφορά στην σύγκριση του A και του B μέρους της άσκησης, προφανώς η οργάνωση των δεδομένων σε ένα αρχείο στον δίσκο ήταν προτιμότερη, εφόσον κάθε φορά χρειαζόταν να γίνει επεξεργασία μόνο 256 bytes στη μνήμη. Αντίθετα, έχοντας όλα τα δεδομένα στη μνήμη δεν γίνεται σωστή κατανομή πόρων και επομένως μειώνεται η απόδοση του προγράμματος.

Σύμφωνα με τη θεωρία, η πολυπλοκότητα για τις λίστες στη μνήμη είναι περίπου γραμμική $O(n \cdot 3)$, εφόσον για την κάθε αναζήτηση γίνονται πλην των 2 αρχικών, 3 συγκρίσεις/αναθέσεις με κάθε στοιχείο της λίστας. Όταν τα δεδομένα κατανέμονται σε 100 λίστες με τη μέθοδο κατακερματισμού, η πολυπλοκότητα θα γίνει $O((n \cdot 3)/100)$. Αντίστοιχα, για τις λίστες στον δίσκο, εφόσον η μία πρόσβαση (read) φέρνει στη μνήμη 32 στοιχεία για αναζήτηση, η πολυπλοκότητα θα είναι $O(n/32)$, ενώ για τον κατακερματισμό στον δίσκο $O((n/32)/100)$. Όπως φαίνεται και με περαιτέρω πράξεις, οι αριθμοί που έχουν προκύψει συνάδουν με τη θεωρία.

Επιπλέον, σε κάθε δομή παρατηρείται ότι για τις αποτυχημένες αναζητήσεις απαιτείται ο διπλάσιος αριθμός συγκρίσεων/προσβάσεων από ότι για τις τυχαίες σε εύρος επιτυχημένες. Αυτό επιβεβαιώνεται και από τη θεωρία, καθώς για n εισαγωγές στοιχείων η πολυπλοκότητα $O(n)$ είναι η διπλάσια από τον μέσο όρο των επιτυχημένων αναζητήσεων.

Πηγές:

- Φροντιστήριο μαθήματος
- Ιστοσελίδες όπως οι oracle και stackoverflow, με μερικά παραδείγματα:

<https://docs.oracle.com/javase/7/docs/api/java/io/RandomAccessFile.html>

<https://javadeveloperzone.com/java-basic/java-convert-int-to-byte-array/>

<https://stackoverflow.com/questions/11437203/how-to-convert-a-byte-array-to-an-int-array>